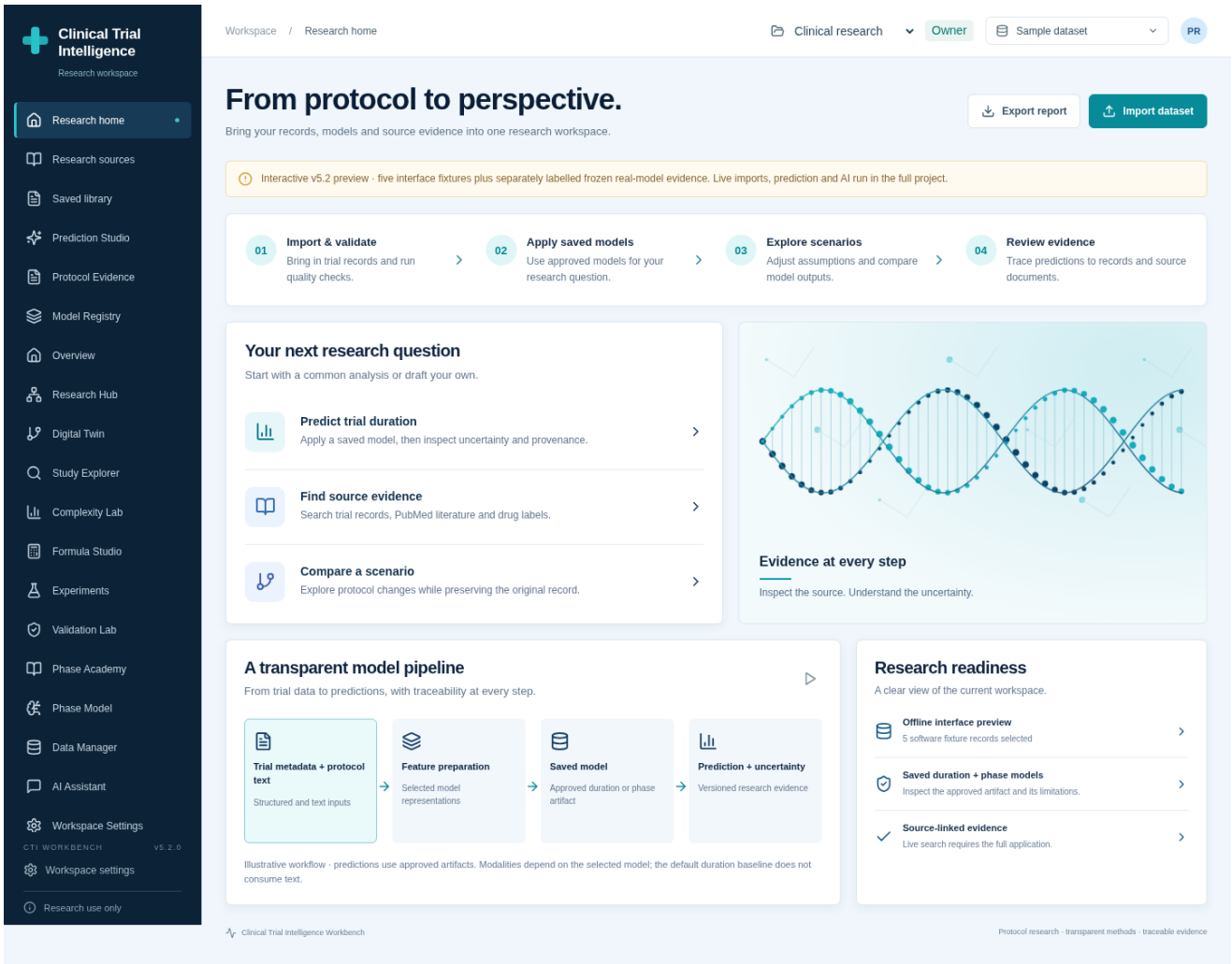
Clinical Trial Intelligence

Workbench v5.2.0

End-to-end operating guide

Install · Run · Import · Predict · Review · Train · Deploy

11 October 2026



Research decision support. Source records, assumptions and model evidence remain distinct.

This guide describes the delivered development release. Live services and production deployment require operator configuration.

1. What this release does

Clinical Trial Intelligence helps researchers inspect registry records and protocols, calculate transparent protocol complexity, apply saved duration and phase models, explore hypothetical scenarios, and preserve source evidence. This is research decision support. Trial duration, trial completion, trial phase and clinical success are different outcomes.

The normal workflow is: create an account and project, import records, validate the source, apply an approved saved model, review uncertainty and limitations, compare scenarios, collect source citations, and export a report. Training is a separate owner-controlled workflow.

The release was built from the selected Git repository's v5 source. The supplied prompt and four PDFs informed the changes. The uploaded v5.1 ZIP exceeded the execution tool's 32 MiB transfer limit and could not be inspected or merged. No claim is made that v5.1-only changes are included. The shared ChatGPT conversation and unavailable third-party plugins were not used as source evidence.

New in v5.2.0

Area	Delivered behavior
Research home	Responsive navy/teal workspace, guided workflow, interactive decorative helix and model-pipeline illustration
Research sources	ClinicalTrials.gov trial search, PubMed publication metadata, openFDA drug-label search, fixed HTTPS destinations and bounded responses
Saved evidence	Persistent project library, source links, retrieval timestamps, CSV export and role checks; new saves use server-cached source results
Account verification	Operator-configured Resend delivery, hashed one-hour tokens, explicit confirmation, single use, rate limits and optional required-verification gate
Cohort monitoring	Reproducible JSON report of missingness, feature shift, unseen categories, identifier overlap and input hashes
Existing research	Saved-model inference, protocol evidence, phase education, Digital Twin, model registry, roles, audit history and provider selection retained

The assistant supports genuine server-sent provider text, Markdown/tables and safe source links. The release verification report records the final tests. External source access, real email delivery, paid AI responses, remote PostgreSQL and deployment require the operator's configuration. A mocked integration test is not proof of live service access.

Requested capabilities that need further evidence or infrastructure

A clinical-success predictor, recruitment-risk model and amendment-risk model require defined labels, suitable historical feature snapshots and independent validation; no probabilities for these outcomes have been invented. The bundled cohort contains 800 records, not hundreds of thousands. The separately recovered 1,800-record acquisition is incomplete and was not used to fit the approved artifacts.

Google login/SSO, an external durable job queue, managed object storage, large-scale distributed training and regulatory qualification are separate deployment projects. Existing owner/researcher/reviewer/viewer permissions remain authoritative. Adding JWTs or a hash chain alone would not establish HIPAA, GDPR or 21 CFR Part 11 compliance. No software or training process can guarantee mistake-free predictions.

2. Package contents and preview

Extract the ZIP into a new directory named Clinical-Trial-Intelligence-Workbench-v5.2.0. Keep your previous installation and its runtime data as a backup. Do not copy a new .env.example over a configured .env.

The archive includes backend and frontend source, built frontend/dist, approved model artifacts and manifests, frozen public source data, tests, scripts, configuration templates, documentation and a self-contained preview under preview/.

Open preview/Clinical-Trial-Workbench-Preview.html to explore the interface without installing Python. This preview uses labelled software fixtures and frozen evidence. It cannot authenticate users, send email, search live sources, train Python models or call an AI provider. Its local draft behavior is not server persistence. Run the full application for these capabilities.

The screenshots under docs/v52-evidence show the rendered application, not a proposed design. The editable guide is docs/V5_2_FULL_GUIDE.md; the PDF is generated from this source. Historical v4/v5 manuals remain available for detailed methods and version context.

3. Quick start on Windows

Install Python 3.12. The ZIP includes the built UI, so Node is only needed for frontend changes.

1. Extract the archive into a writable folder, for example P:\Final_Year_Project\Clinical-Trial-Intelligence-Workbench-v5.2.0.
2. Open PowerShell in that folder and run the setup launcher.
3. Start the app, leave the terminal open and open the loopback address printed by the launcher on your own computer.
4. Open the local .env in a private text editor, copy SETUP_TOKEN into the first-owner form, and create your account with a password of at least 12 characters.
5. Create a project, load the starter cohort from Data Manager, and open Prediction Studio.

```
py -3.12 scripts\setup.py --dev
.\.venv\Scripts\python.exe scripts\start.py
```

The existing SETUP_WINDOWS.bat and START_WINDOWS.bat launchers call these supported scripts. Never paste SETUP_TOKEN or provider keys into chat, a repository, screenshots or shared documents. Each installation should generate its own .env during setup. The release archive excludes the local .env and account database.

4. Quick start on Linux and macOS

```
python3.12 scripts/setup.py --dev
.venv/bin/python scripts/start.py
```

Run from the extracted project root. The setup creates .venv, installs pinned core/test dependencies, generates a private missing setup token and preserves existing configuration. The server defaults to loopback port 8000 and local SQLite storage. Stop it with Ctrl+C. Use the .venv interpreter for every research command.

For a fresh Git checkout without frontend/dist, install Node 22 or a compatible supported Node runtime and build the UI first:

```
cd frontend
npm ci
npm run build
cd ..
python3.12 scripts/setup.py --dev
.venv/bin/python scripts/start.py
```

Node 24.19.0 and Python 3.12.14 were available in the build environment. CI specifies Node 22 and Python 3.12. npm ci preserves the lockfile. To edit the UI, run npm run dev inside frontend while the Python server runs on port 8000; Vite proxies /api to the backend.

5. First account, projects and roles

The first-owner setup is single use. Later team members join using owner-created invitation tokens from Workspace Settings. These invitation tokens are separate from email-verification links. Existing invitations are manually shared; automatic invitation mail is not claimed.

Role	Main permissions
Owner	Import, predict, train candidates, approve/activate models, manage members/settings and inspect audit history
Researcher	Read, import, predict, save scenarios/source evidence, use own AI conversations; cannot promote models
Reviewer	Read evidence and perform supported protocol review; no data import, paid AI or source-library writes
Viewer	Read permitted project evidence; no mutation, training or paid AI calls

Choose the active project in the top bar. Datasets, saved evidence and inference records belong to that project. AI conversations additionally belong to their creating user. Roles are enforced by the server, not merely by hidden buttons. Use separate test accounts to verify the permissions expected in your deployment.

6. Email verification

The application can send a verification link through Resend when the operator configures a real sender domain and server-side API key. No email credentials or verified sender are bundled.

```
MAIL_API_KEY=
MAIL_FROM=
PUBLIC_BASE_URL=
REQUIRE_EMAIL_VERIFICATION=false
```

Enter the API key securely in your server environment. MAIL_FROM should be an address from your verified sending domain, optionally with a display name. PUBLIC_BASE_URL must be the actual HTTPS origin of your application, without a trailing path. Keep REQUIRE_EMAIL_VERIFICATION=false for an unconfigured local installation. Enable it only after testing delivery and the HTTPS origin.

In Workspace Settings, choose Send verification email. The recipient opens the link and explicitly confirms it. The link token is held in the URL fragment, cleared from the address bar and sent in a POST body. It expires after one hour; a new request revokes earlier outstanding tokens. The database stores hashes, not the raw tokens. Confirmation does not log an anonymous person in. The requester must subsequently authenticate.

When verification is required, an unverified signed-in account sees the verification gate even before creating its first project. Existing accounts are not silently treated as verified. Email outages produce an explicit unavailable message. The tests mock delivery and never send messages to real people.

7. Import real data and preserve provenance

Open Data Manager and select Load starter cohort for the bundled authentic 800-record source. Importing copies the source into the active project; it does not fit a model. The five-record sample is only for software demonstrations.

Supported local inputs include CSV, Parquet, Excel and flat JSON. Additional Imports supports pasted CSV/JSON and approved HTTPS URLs. The existing ClinicalTrials.gov importer produces canonical trial records; the new Research sources screen searches citations without automatically ingesting them into a training dataset.

Every canonical record needs nct_id and title. Keep identifiers unique. Preserve missing values rather than filling them with invented zeros. Keep actual versus estimated dates and date precision intact. Store source snapshot date, acquisition URL and file hashes. Do not change raw AACT files.

For frozen AACT data, use the supported feature extraction command and a new output directory:

```
.venv/bin/python scripts/build_features.py \
  --input data/raw/aact_snapshot \
  --snapshot-date 2026-10-01 \
  --output data/processed/aact-2026-10-01
```

Use your actual snapshot date. This example date is a command example, not a statement that such an acquisition is bundled. A large snapshot may exceed the default 100,000-row import or 20,000-row training limits; assess memory, read the pipeline assumptions and configure bounded limits deliberately.

8. Predict without retraining

1. Select the imported dataset in the top bar.
2. Open Prediction Studio and choose duration/prolonged-duration or phase consistency.
3. Use the active approved model. Start with retrospective research screening for latest-state public records.
4. Run prediction, then inspect missing features, warnings, uncertainty, model branch and input/model hashes.
5. Save or export the result batch and reopen it after a refresh.
6. For prospective questions, use strict pretrial availability only with authentic feature timestamps from appropriate historical sources.

Strict mode blocks unknown or late feature availability. Do not invent timestamps to make it pass. Retrospective screening does not establish prospective predictive ability. The default duration champion is a simple median baseline selected on validation evidence; identical duration estimates across records can be correct behavior.

The approved duration artifact contains nine experiment branches; the phase artifact contains two. Current verification reproduces saved outputs and metrics without model fitting. It does not establish clinical utility, external generalization or a new scientific benchmark.

9. Read the research screens

Screen	Purpose and interpretation
Research home	Guided entry to import, prediction, scenarios and evidence; pipeline and helix visuals explain navigation
Prediction Studio	Apply approved saved artifacts and inspect uncertainty, availability checks and persisted batches
Model Registry	Review hashes, model cards, approvals, active versions and rollback choices
Protocol Evidence	Upload supported PDF/text, extract source passages, review them and compare versions
Overview / Study Explorer	Inspect records, filter/search, compare up to three studies and export descriptive data
Complexity Lab / Formula Studio	Transparent PCI calculations and weights; PCI is descriptive complexity, not clinical success
Research Hub	Connect a selected trial to available model evidence and similar records
Digital Twin	Compare an explicit hypothetical protocol with its preserved source
Experiments / Phase Model	Owner-only candidate training and phase classifier research
Validation Lab	Inspect temporal splits, held-out evidence, calibration and limitations
Phase Academy	Educational phase objectives and trial journey; not physiological simulation
Research sources / Saved library	Search public sources and collect source-linked project evidence
AI Assistant	Provider-configured research conversations with bounded project context and saved history
Workspace Settings	Projects, roles, assistant preferences, usage, audit history and email verification

10. Digital Twin and animations

Select a source study, change permitted protocol fields, enter site/recruitment/startup/follow-up assumptions, then choose Compute & save scenario. The result preserves the original record, hypothetical changes, descriptive PCI and available model output.

Planning months are calculated from explicit assumptions. They are not predicted stage durations or a learned recruitment-risk score. An input change may leave a median-baseline model unchanged. Model associations and scenario differences do not prove causal effects.

The trial journey, phase learning screens and pipeline animation have play/pause or selection controls. The molecular helix responds to pointer movement and respects reduced-motion preferences. The pipeline shows structured and text inputs conceptually; actual modality usage depends on the selected artifact. The default duration baseline does not consume protocol text. The new explanation panel animates actual numeric feature contributions or global importances when the saved model supplies them. Constant baselines explicitly report no feature attribution. Existing attribution evidence remains labelled by its method; no decorative animation is presented as a SHAP result.

11. Search and save source evidence

Select ClinicalTrials.gov, PubMed or openFDA; enter a research query and search. The response is limited to 10 visible results. The API accepts 1–20 records per request, bounded query length and per-project/user request limits.

ClinicalTrials.gov returns registered trial metadata. PubMed returns publication metadata and journal/author context; this connector does not claim to retrieve full-text papers or abstracts. openFDA currently searches drug labels. A label is not an adverse-event incidence estimate, and the presence of a warning does not establish causality for a trial.

Open the original HTTPS source link. Save useful records to the active project library. New bookmark saves must match an unexpired server-cached result from that project's search; submitted title/summary edits cannot override the canonical cached evidence. Results are retained in the search cache for 24 hours; repeat a search if an unsaved result has expired. Each project can retain 200 bookmarks.

Read-only roles can view/search; owners and researchers can save/remove. Export CSV to preserve citations and retrieval timestamps. Network outages and malformed upstream responses are shown as unavailable, not as successful searches with invented fallback data.

Required hosts are clinicaltrials.gov, eutils.ncbi.nlm.nih.gov and api.fda.gov. The exact hosts were saved in the cloud configuration draft. After rechecking, all three live connectors returned cited records successfully; the release includes the retrieval receipt. Review/save the environment settings and publish to retain this configuration for future tasks.

12. Configure the research assistant

Configure one or more provider keys and account-supported model IDs in server-only .env or your deployment's secret store. OpenAI, Anthropic, Gemini and an explicitly configured OpenAI-compatible provider are supported. There is no free Emergent account/key bundled with this release.

```
AI_PROVIDER=auto
AI_PROVIDER_PRIORITY=anthropic,openai,gemini,compatible
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=
OPENAI_API_KEY=
OPENAI_MODEL=
GEMINI_API_KEY=
GEMINI_MODEL=
COMPATIBLE_API_KEY=
COMPATIBLE_BASE_URL=
COMPATIBLE_MODEL=
```

Use model IDs that your provider actually exposes; names suggested in the supplied PDFs are not access guarantees. Restart after changing server configuration. The UI shows configuration presence, which is different from tested account access. No paid provider request was made during this release validation.

Choose a dataset/study and a fast, balanced or deep response profile. Profiles control bounded context and output size, not measured model quality. Your question, bounded history and selected project evidence are sent to the chosen provider. Do not submit private patient information without an appropriate authorized deployment/data policy.

Saved context reference cards identify the source material supplied. They do not certify every generated sentence. Verify claims against the source. Provider failures preserve the draft/history state and show an explicit error. The browser receives actual provider text deltas via SSE, then replaces the partial display with the server-confirmed saved message. Stop reply cancels the request. Partial text remains visibly unconfirmed after a failure and is not presented as a completed saved answer. Streaming selects one configured provider; it does not switch providers midway through an answer. Paid live provider access remains unverified.

13. Train duration and phase candidates

Training requires authentic cohorts with suitable outcomes. Completed-trial status is not clinical efficacy or trial success. Do not train a success classifier from arbitrary status proxies and label its probability clinical success.

The duration pipeline uses completed actual day-precision start/completion dates, temporal splitting and purging. Preprocessing is fitted on the training partition. Separate validation/calibration/test partitions preserve their purpose. Keep structured-only, text-only and multimodal comparisons on the same eligible cohort and splits. Compare advanced models against simple baselines.

Run into new directories; never overwrite examples/starter-duration-v5 or examples/starter-phase-v5:

```
.venv/bin/python scripts/train_models.py \
  --input data/starter/trials.csv \
  --metadata data/starter/metadata.json \
  --config configs/research.json \
  --output results/duration-candidate-001

.venv/bin/python scripts/train_phase_model.py \
  --input data/starter/trials.csv \
  --metadata data/starter/metadata.json \
  --output results/phase-candidate-001
```

On Windows use the .venv\Scripts\python.exe path and enter each command on one line or use PowerShell backticks for continuation. The sample commands use the real bundled cohort to make the workflow reproducible; they do not create a larger benchmark.

CLI output is a standalone research experiment. For application-managed candidates, train through the owner's Experiments or Phase Model screen so the run is registered to that project. Review the manifest, source hash, split counts, metrics and warnings before research approval and activation. Retain the previous approved version for rollback.

Optional extensions are separate: requirements-shap.txt adds SHAP; requirements-research.txt adds transformer, PyTorch and survival packages. Install them only for those experiments. A skipped optional SHAP test is distinct from a failure in core saved-model inference. GPU-heavy embedding work was not prepared or measured in this release.

14. Improve models over time responsibly

Daily ingestion does not justify daily automatic promotion. Preserve a new dated raw snapshot, validate identifiers/fields/timestamps, compare distributions, train a separate candidate, evaluate on untouched later cohorts, and require review before activation. Keep a model registry record with code/config/environment/data/model hashes.

The new monitor command provides an operational comparison:

```
.venv/bin/python scripts/monitor_cohort.py \
  --reference data/starter/trials.csv \
  --current data/clinicaltrials-v5-recovered/trials.csv \
  --output results/cohort-monitoring-001.json
```

It computes missingness, medians, numeric population stability index (PSI), values outside the reference range, categorical total-variation distance, unseen categories, duplicate identifiers and cohort overlap. Both files remain unchanged. Outputs include their hashes. Empty cohorts are refused. Unknown/nonfinite numeric values stay missing.

PSI >= 0.2, any out-of-range value or a missingness difference >= 0.1 flags a review. These are engineering heuristics, not clinical thresholds. Small cohorts and shared trial identifiers require special care. Feature drift cannot measure prediction error without later observed outcomes. The bundled 800 versus 1,800 comparison requires review and is included as an example of real measured drift, not independent validation.

For hundreds of thousands of records, use curated AACT snapshots or authorized rate-limited registry acquisition, immutable storage, validated joins, dated feature snapshots, a durable worker queue and resource budgets. Start with representative slices. Measure peak RAM/CPU/disk before increasing training limits. No 500,000-record training or claim of better accuracy was performed here.

15. Reports, exports and scientific interpretation

Use Study Explorer for descriptive CSV and the top-level Export report control for printable HTML. Open the downloaded report and use the browser's Print to PDF feature. Prediction Studio exports saved evidence; Digital Twin exports scenario JSON; Saved library exports citations as CSV. Keep model and dataset hashes with any result you share.

Separate measured outputs, uncertainty, planning assumptions, software fixtures and educational illustrations. Report the small held-out sample sizes and retrospective source limitations. SHAP and other feature attributions are associations, not causal explanations. Keep source-linked protocol passages accessible for human review.

16. Deployment and backups

Local SQLite is a development fallback. A production deployment requires managed PostgreSQL with verified TLS, persistent artifact storage, an HTTPS reverse proxy, secret management, backups, monitoring, rate limits and an operator-reviewed access policy. The application rejects production local auth/SQLite configurations.

The repository includes a Dockerfile and local Docker Compose configuration. These were not built or deployed in this environment. Compose defaults bind the published port to loopback. Do not describe the local compose profile as a complete production stack. Set DATABASE_URL to your approved PostgreSQL endpoint with TLS; prefer verify-full and a trusted CA when supported.

Back up the private .env through your secret-management process, the metadata database and the full artifact volume together. Test restoration into an isolated copy. Do not reset or overwrite a production workbench during upgrade. The app's existing in-process training worker marks interrupted jobs failed after restart; it does not offer durable distributed job recovery.

Keep AUTH_MODE=session and use real accounts for shared access. Enable email verification only after delivery works. HTTPS, domain configuration and sender verification occur outside this ZIP. Regulatory compliance requires organizational controls and validation beyond the application's features.

17. Testing and troubleshooting

Run from the project root after setup:

```
.venv/bin/python -m pip check
.venv/bin/python -m pytest -q
.venv/bin/python scripts/run_smoke_test.py
.venv/bin/python scripts/verify_v5_models.py
.venv/bin/python scripts/verify_release.py
```

verify_release.py is for an extracted packaged release. It checks packaged bytes against SHA256SUMS.txt, not publisher authenticity. Editing/rebuilding a packaged file can legitimately invalidate its original checksum; do not change expected hashes to hide an integrity failure. Build a fresh release manifest using the package script when intentionally releasing changed code.

Symptom	Next check
Python version refused	Run setup with Python 3.12 and use the generated .venv interpreter
UI missing	npm ci and npm run build in frontend; rerun setup
Owner setup denied	Use this installation's private SETUP_TOKEN; first-owner setup cannot be repeated after an account exists
Verification blocked	Configure verified sender, MAIL_API_KEY and actual HTTPS PUBLIC_BASE_URL; inspect safe server errors
Research source unavailable	Check exact host allowlists/proxy policy; retry after configuration changes
AI unavailable	Check provider/model/account access, quota and network policy; never put keys in frontend code
Strict inference blocked	Supply authentic historical feature availability evidence; do not invent timestamps
Same duration for several trials	Inspect active branch; the validation-selected median baseline can be constant
Missing scientific risk category	No approved model exists for that outcome; leave it unavailable
Database/artifact missing after restart	Restore matching persistent metadata and artifacts from tested backups

18. Build, package and reproduce this release

```
cd frontend
npm ci
npm run build
VITE_DEMO_MODE=true npm run build -- --outDir dist-preview
cd ..
.venv/bin/python scripts/build_preview.py
.venv/bin/python scripts/package_release.py \
  --version 5.2.0 --output /path/outside/project/CTI-v5.2.0.zip
```

PowerShell preview build:

```
cd frontend
$env:VITE_DEMO_MODE='true'
npm run build -- --outDir dist-preview
Remove-Item Env:VITE_DEMO_MODE
cd ..
.\.venv\Scripts\python.exe scripts\build_preview.py
```

To regenerate the PDF, install requirements-docs.txt into your environment and run scripts/build_v52_guide.py. Build the normal frontend without VITE_DEMO_MODE for the full application. The package script includes frontend/dist and excludes node_modules, virtual environments, .env, runtime stores, old ZIPs and caches. It writes one fresh checksum manifest and tests archive integrity. Unpack into a clean directory and run verify_release.py before distribution.

The cloud environment install script prepares the dependencies and UI; start instructions launch the app and verify health/bootstrap. Each cloud task already has an isolated checkout; no extra Git worktree is needed. Processes do not survive environment snapshots and must restart. Saving configuration is separate from reviewing/saving and publishing the environment.